Universidad Icesi

De "Vibe Coding" a Ingeniería Guiada por Especificaciones

Las 6 Fases de Spec-Driven Development (SDD), Plantillas y Comandos /speckit.*

Computación en Internet II • Semana 13 • Universidad Icesi

Universidad ICESI

Ruta de Aprendizaje — Sesión de 2 Horas (5 Micro-Ciclos)

Parte I: Las 6 Fases SDD y Generación en Vivo desde Plantillas

Ciclo 1 (20 min) • Fase 1: `Constitution` y Memoria
SDLC Tradicional vs. las 6 Fases de SDD. Inspección de `00-constitution.md`, `.agents/memory/` y `specs/templates/`.

Ciclo 2 (25 min) • Fases 2 y 3: `/speckit.specify` + `/speckit.clarify`
Creación en vivo de `001-cursos-rest-api/01-spec.md` desde el template e **Interrogatorio Socrático** de casos borde.

Ciclo 3 (25 min) • Fases 4 y 5: `/speckit.plan` + `/speckit.tasks`
Auditoría del código existente para generar `02-plan.md` (CÓMO técnico) y partición atómica en `03-tasks.md`.

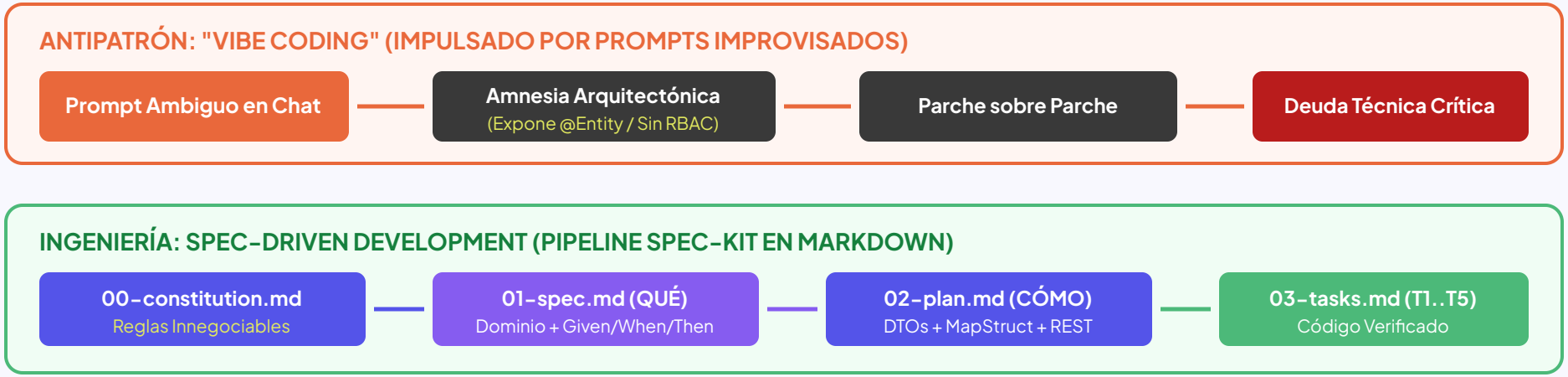
Parte II: Single-Task Rule, Validación y Comandos de Agente

Ciclo 4 (30 min) • Fase 6: `/speckit.implement` (T1 a T4)
Regla de una sola tarea a la vez para construir `/api/v1/cursos` validando con `./mvnw test-compile` en cada paso.

Ciclo 5 (20 min) • Validación Cruzada (T5) y Reto Autónomo
Pruebas en Postman contra `01-spec.md` y anatomía de `.github/prompts/` (`/speckit.*`) y `.agents/skills/`.

Flujo Real en Clase: Partimos **solo con las plantillas en blanco** (`specs/templates/`) y generamos `001-cursos-rest-api` en vivo refinando las decisiones como arquitectos.

01. La Crisis del "Vibe Coding" vs. SDD



Experimento de Choque: "¿Me haces el CRUD REST de Cursos?"

Lo que genera la IA con un Prompt de "Vibe Coding":

```java
@RestController
@RequestMapping("/cursos") // 1. Rompe prefijo /api/v1/cursos
public class CursoRestController {

    @Autowired // 2. Inyección por atributo prohibida
    private CursoRepository cursoRepository; // Salta el Service

    @GetMapping("/listarCursos") // 3. URI estilo RPC no RESTful
    public List<Curso> listar() {
        // 4. PELIGRO CRÍTICO: Retorna @Entity Curso directo
        // Filtra Curso -> Profesor -> Usuario.password (BCrypt)
        // y desata ciclos infinitos en Jackson
        return cursoRepository.findAll();
    }

    @PostMapping("/crear")
    // 5. Sin @PreAuthorize ni validación de rol/autoridad
    public Curso crear(@RequestBody Curso curso) {
        return cursoRepository.save(curso);
    }
}
```

Las 4 Fallas Estructurales del "Vibe Coding":

1. Amnesia Arquitectónica (Context Drift)

El historial del chat es efímero: la IA olvida que nuestro proyecto usa `/api/v1/...`, Lombok y MapStruct.

2. Fuga de Entidades JPA y Secretos

Al retornar `Curso` sin DTO, serializa el objeto `Profesor` y el `password` de su `Usuario`.

3. Ceguera frente al Modelo RBAC Real

Desconoce que en `data.sql` existen permisos específicos (`COURSE_READ`, `COURSE_WRITE`).

4. Acoplamiento Prematuro

Mezcla reglas de negocio con código en un solo paso sin definir criterios de aceptación verificables.

Arquitectura SDD Zero-Install

¿Qué es GitHub Spec-Kit (`github/spec-kit`)? Un enfoque de **Spec-Driven Development (SDD)** donde el código es el resultado compilado de especificaciones Markdown. No necesitamos instalar `uv` : disponemos de plantillas, memoria y comandos directamente en el repo.

1. Punto de Partida Limpio: Solo Plantillas y Memoria

Al iniciar la clase, la carpeta `001-cursos-rest-api/` **aún no existe**. El repositorio solo tiene `00-constitution.md` , `.agents/memory/` y los moldes en blanco en `specs/templates/` .

2. Generación en Vivo + Refinamiento del Desarrollador

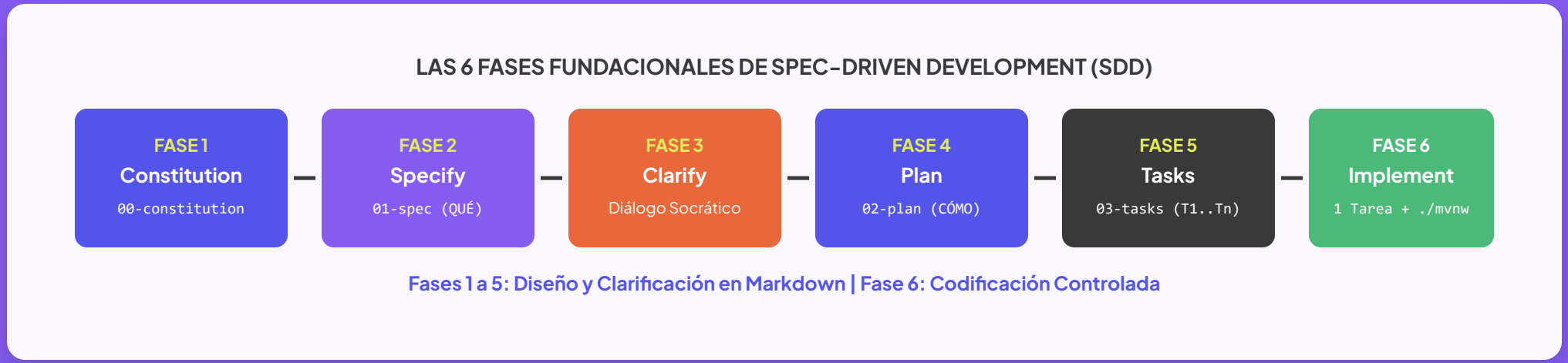
Con lenguaje natural e invocando `/speckit.*` , el agente crea `001-cursos-rest-api/` desde las plantillas y nosotros auditamos y refinamos cada decisión.

3. Doble Integración: Slash Commands y Skills

Funciona con `/speckit.*` en VS Code / Copilot (`.github/prompts/`) y con skills nativos en Antigravity CLI (`.agents/skills/`).



02. SDLC Tradicional vs. Las 6 Fases Fundacionales de SDD



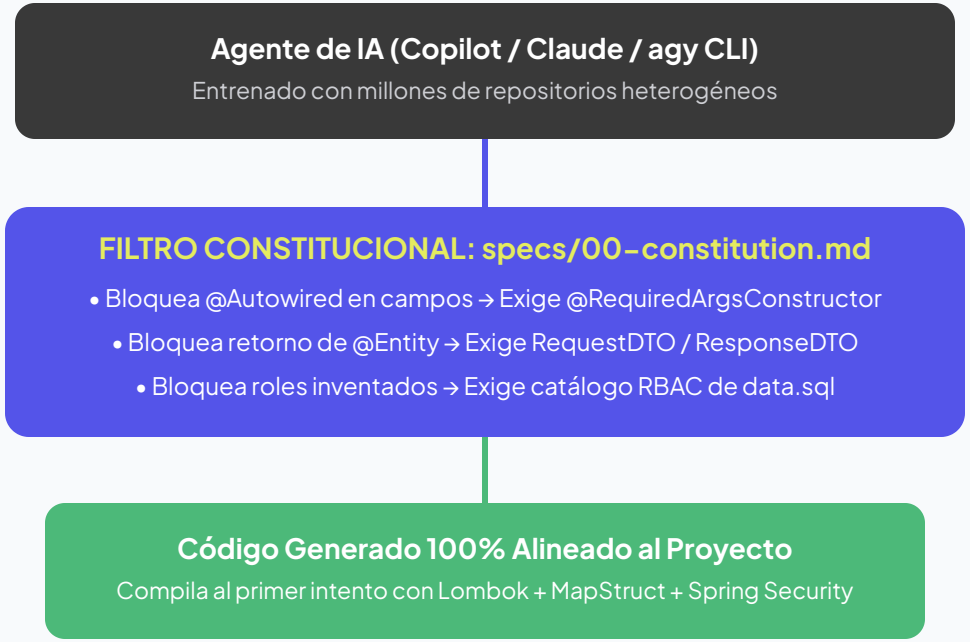
Universidad ICESI

SDLC Tradicional / Vibe Coding vs. Las 6 Fases de SDD

Fase SDD	Comando Slash / Skill	SDLC Tradicional / Vibe Coding	Spec-Driven Development (SDD)	Entregable en Git
1. Constitution	`/speckit.constitution`	Requisitos informales; la IA ignora convenciones del proyecto.	**Reglas de Oro innegociables** y memoria permanente del stack.	`00-constitution.md` + `.agents/memory/`
2. Specify	`/speckit.specify`	Análisis ambiguo mezclando QUÉ con código prematuro.	El **QUÉ y POR QUÉ** funcional con escenarios **Gherkin**.	`01-spec.md` *(Borrador inicial)*
3. Clarify	`/speckit.clarify`	Diseño reactivo: la IA adivina casos borde en silencio.	**Diálogo Socrático**: 3 a 5 preguntas críticas sobre casos borde.	`01-spec.md` *(Clarificado)*
4. Plan	`/speckit.plan`	Codificación temprana duplicando servicios o exponiendo `@Entity`.	Auditoría de código existente y diseño del **CÓMO** técnico.	`02-plan.md`
5. Tasks	`/speckit.tasks`	Implementación monolítica imposible de depurar.	Partición en **tareas atómicas secuenciales** (`T1..T5`).	`03-tasks.md` `[ ]`
6. Implement	`/speckit.implement`	Pruebas tardías y alta deuda técnica.	**Single-Task Rule**: 1 tarea a la vez + `./mvnw` + Postman.	Código Java + `03-tasks.md` `[x]`

Diferencia decisiva: En SDD las Fases 1 a 5 ocurren **sin escribir una sola línea de Java**; recién en la Fase 6 el agente codifica bajo contrato estricto.

Pilar 1: ¿Por qué existe 00-constitution.md?



Los 5 Artículos de nuestra Constitución en `springboot`:

Art. 1 • Stack Tecnológico Inalterable
Java 21, Spring Boot 3.4.3, H2, JJWT 0.12.6, Lombok y MapStruct 1.6.3.

Art. 2 • Fronteras Estrictas de Paquetes
Separación entre `controller.mvc`, `controller.rest`, `dto`, `mapper` y `service`.

Art. 3 • Inyección por Constructor y Aislamiento DTO
Obligatorio `private final` + `@RequiredArgsConstructor` y uso de `ResponseEntity<T>`.

Art. 4 y 5 • Seguridad RBAC y Verificación Maven
Uso obligatorio de `@PreAuthorize` y validación con `./mvnw clean compile`.

Inspección de specs/00-constitution.md (Artículos 1 al 3)

Extracto Real de `springboot/specs/00-constitution.md`:

```
## Artículo 3. Estándares de Codificación y DTOs

1. Inyección por Constructor Obligatoria:
   - PROHIBIDO usar @Autowired sobre atributos.
   - Toda dependencia debe ser 'private final' con
     '@RequiredArgsConstructor' de Lombok.

2. Aislamiento con Patrón DTO:
   - Ningún método en 'controller.rest' puede recibir
     ni retornar una entidad del paquete 'model'
     (Usuario, Curso, Profesor, Matricula).
   - Separación explícita entre *RequestDTO y *ResponseDTO.

3. Semántica RESTful y ResponseEntity<T>:
   - URIs con sustantivos en plural (/api/v1/cursos).
   - POST retorna 201 Created + Header Location.
   - DELETE retorna 204 No Content.
```

Mapa de Fronteras de Paquetes (Artículo 2):

Paquete	Permitido	Prohibición Estricta
`controller.rest`	`@RestController` bajo `/api/v1/...`	Inyectar `@Repository` o exponer `@Entity`
`dto.<Modulo>`	POJOs con Lombok (`@Data` , `@Builder`)	Anotaciones JPA (`@Entity` , `@ManyToOne`)
`mapper`	`@Mapper(componentModel = "spring")`	Reflexión en runtime (`ModelMapper`)
`service`	Lógica de dominio y transacciones	Acoplar `ResponseEntity` o `HttpServletRequest`

Efecto inmediato: Al incluir esta tabla en `00-constitution.md` , la IA adopta exactamente el mismo estilo de `UsuarioRestController.java` .

Universidad ICESI

- Cadena Dual Intacta
- Catálogo Real data.sql
- @PreAuthorize Obligatorio
- Verificación ./mvnw

Gobernanza de Seguridad RBAC

Artículo 4 de `00-constitution.md` : Anclaje a `data.sql`

```
## Artículo 4. Reglas de Seguridad y Autorización (RBAC)

1. Protección con @PreAuthorize:
   - Todo endpoint en 'controller.rest' (excepto /api/v1/auth/**)
     debe declarar @PreAuthorize usando hasRole(...) o hasAuthority(...).

2. Catálogo Oficial de Roles y Autoridades (según data.sql):
   - Roles: ADMINISTRADOR, PROFESOR, ESTUDIANTE.
   - Permisos: USER_CREATE, USER_READ, USER_UPDATE, USER_DELETE,
     COURSE_READ, COURSE_WRITE, ENROLLMENT_WRITE.

3. Preservación de la Cadena Dual (SecurityConfig.java):
   - Prohibido alterar @Order(1) (/api/** Stateless JWT) ni @Order(2) (MVC).
```

¿Por qué es vital el punto 2? Sin la lista explícita de permisos de `data.sql`, los LLMs suelen inventar anotaciones como `@PreAuthorize("hasRole('ADMIN')")` que fallan con `403 Forbidden` en tiempo de ejecución.

03. Del "QUÉ" de Negocio al "CÓMO" Arquitectónico

`01-spec.md`

EL QUÉ Y EL POR QUÉ (Dominio)

- Historias de Usuario (HU-01 a HU-05)
- Matriz de Acceso RBAC por operación
- Escenarios Given / When / Then

Prohibido incluir código Java aquí

Gobernado por `00-constitution`

`02-plan.md`

EL CÓMO TÉCNICO (Arquitectura)

- Auditoría de clases existentes a reutilizar
- Contratos JSON (CursoRequest/ResponseDTO)
- Reglas @Mapping de CursoMapper

Tabla exacta de Endpoints + @PreAuthorize

En Vivo: De Lenguaje Natural a 01-spec.md (/speckit.specify + /speckit.clarify)

Paso 1: Generar `01-spec.md` desde el Template (`/speckit.specify`)

```
/speckit.specify Quiero crear la especificación
001-cursos-rest-api para exponer la gestión de Cursos
como API REST en /api/v1/cursos protegida con JWT.
Necesitamos listar todos los cursos, filtrar por
departamento, obtener por ID, crear curso asociándolo
a un profesor existente, actualizar y eliminar.
```

¿Qué hace el Agente automáticamente?

Lee `00-constitution.md` y `specs/templates/01-spec-template.md`, y crea la carpeta `specs/001-cursos-rest-api/01-spec.md` con historias y escenarios Gherkin.

¿Pasamos directo al código? ¡No! El primer borrador aún tiene casos borde ocultos que debemos descubrir con `/speckit.clarify`.

Paso 2: Diálogo Socrático en Fase 3 (`/speckit.clarify`)

Pregunta 1 del Agente: Privacidad del Profesor

"Profesor tiene relación con Usuario (que incluye password). ¿Qué campos exactos del profesor se exponen en el JSON?" → **Decisión:** Aplanar solo `profesorId`, `nombre` y `email`.

Pregunta 2 del Agente: Filtro por Departamento

"¿El filtro ?departamento= distingue mayúsculas y qué retorna si no hay coincidencias?" → **Decisión:** `IgnoreCase` y `200 OK` con `[]`.

Pregunta 3 del Agente: Validaciones y Autoridades

"¿Qué ocurre si creditos <= 0 o profesorId no existe?" → **Decisión:** `400 Bad Request` para validaciones y `404 Not Found` si el profesor no existe.

Caso 001-cursos-rest-api: 01-spec.md en Acción

Escenarios Given / When / Then (`01-spec.md`):

```
### Escenario 2: Rechazo RBAC a Estudiante (403)
- DADO: Que el estudiante apaez@icesi.edu.co posee
  únicamente COURSE_READ y ENROLLMENT_WRITE.
- CUANDO: Intenta registrar un nuevo curso (POST).
- ENTONCES: El sistema bloquea la petición con 403 Forbidden
  antes de ejecutar la lógica de negocio.

### Escenario 3: Creación por Profesor o Admin (201)
- DADO: Un usuario con autoridad COURSE_WRITE y que
  el profesor con id = 3 existe en la BD.
- CUANDO: Envía "Arquitectura de Agentes IA", 3 créditos,
  "Computación y Sistemas Inteligentes", profesorId = 3.
- ENTONCES: Persiste el curso y retorna 201 Created +
  Header Location + representación enriquecida.

### Escenario 4: Profesor Inexistente (404)
- CUANDO: Envía un profesorId = 999 inexistente.
- ENTONCES: Responde 404 Not Found (nunca 500).
```

Matriz RBAC Definida en `01-spec.md` :

Operación de Dominio	Permiso / Rol Exigido	Roles que lo poseen (data.sql)
Listar Cursos / Filtrar Depto.	COURSE_READ	ESTUDIANTE , PROFESOR , ADMINISTRADOR
Consultar Curso por ID	COURSE_READ	ESTUDIANTE , PROFESOR , ADMINISTRADOR
Crear Nuevo Curso	COURSE_WRITE	PROFESOR , ADMINISTRADOR
Actualizar Curso	COURSE_WRITE	PROFESOR , ADMINISTRADOR
Eliminar Curso	ROLE_ADMINISTRADOR	Exclusivo ADMINISTRADOR

Observa cómo la matriz combina **autoridades granulares** (`COURSE_READ` , `COURSE_WRITE`) con **restricción por rol** para el borrado físico.

Pilar 3: 02-plan.md — Reutilización y Aplanado DTO

Paso 1 en `02-plan.md`: Auditar antes de Crear

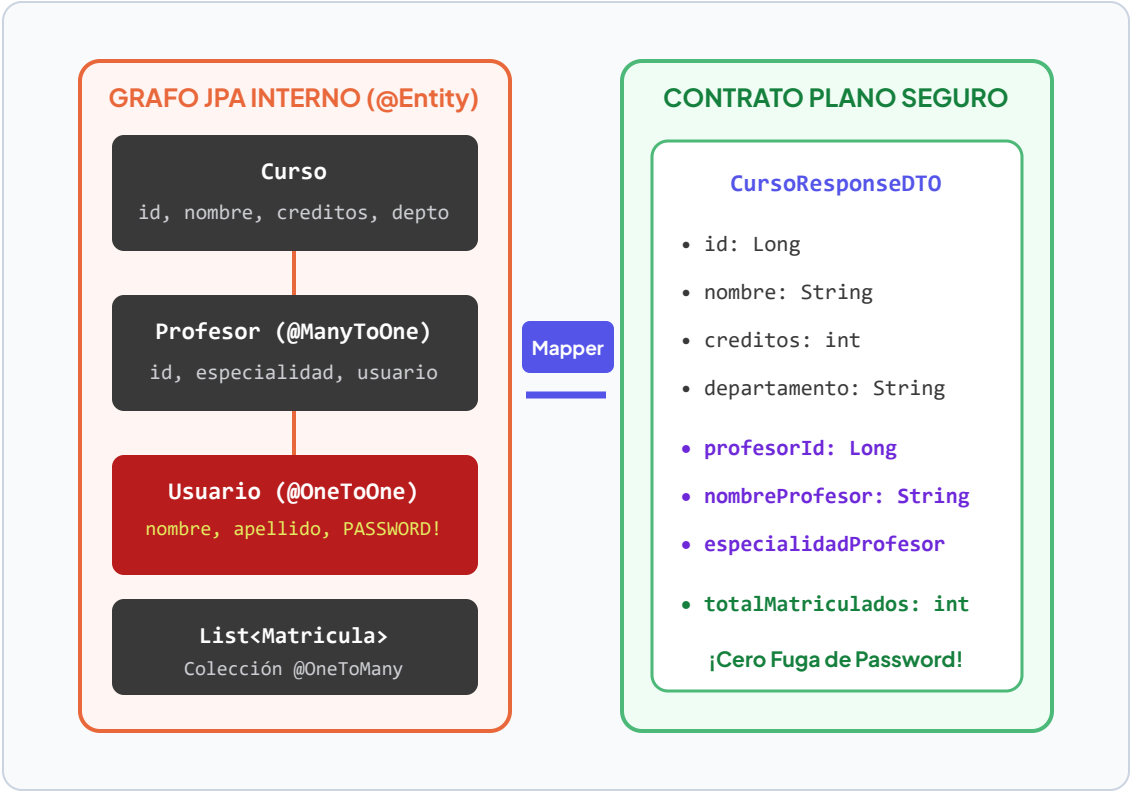
`CursoRepository.java` (Reutilizar 100%)
Ya incluye `findByDepartamentoIgnoreCase(String depto)`.

`CursoService.java` (Solo +1 método puente)
Solo agregamos `listarPorDepartamento(String)` sin romper MVC.

Contrato JSON de Salida (`CursoResponseDTO`):

```
{
  "id": 15,
  "nombre": "Arquitectura de Agentes IA",
  "creditos": 3,
  "departamento": "Computación y Sistemas Inteligentes",
  "profesorId": 3,
  "nombreProfesor": "Alejandro Munoz",
  "especialidadProfesor": "Arquitectura de Software y Patrones",
  "totalMatriculados": 0
}
```

Paso 2 en `02-plan.md`: Diseño del Aplanado (Flattening)



Universidad ICESI

Especificación de Mapeo y Endpoints en 02-plan.md

Sección 3 de `02-plan.md` : Reglas de `CursoMapper`

```
## 3. Estrategia de Mapeo (mapper/CursoMapper.java)

Interfaz @Mapper(componentModel = "spring"):

1. CursoResponseDTO toCursoResponseDTO(Curso curso):
   - @Mapping(source = "profesor.id", target = "profesorId")
   - @Mapping(source = "profesor.especialidad",
              target = "especialidadProfesor")
   - @Mapping(target = "nombreProfesor",
              expression = "java(obtenerNombreCompletoProfesor(curso))")
   - @Mapping(target = "totalMatriculados",
              expression = "java(curso.getMatriculas() != null
                           ? curso.getMatriculas().size() : 0)")

2. Curso toEntity(CursoRequestDTO requestDTO):
   - Ignorar id, profesor y matriculas (ignore = true)
     porque los resuelve CursoService.
```

Sección 4 de `02-plan.md` : Tabla de Endpoints HTTP

Verbo y Ruta	Regla @PreAuthorize	Éxito / Error
`GET /api/v1/cursos` `?departamento=...`	`hasAuthority('COURSE_READ')`	`200 OK`
`GET /api/v1/cursos/{id}`	`hasAuthority('COURSE_READ')`	`200 OK` / `404`
`POST /api/v1/cursos`	`hasAuthority('COURSE_WRITE')`	`201 Created` + `Location` / `404`
`PUT /api/v1/cursos/{id}`	`hasAuthority('COURSE_WRITE')`	`200 OK` / `404`
`DELETE /api/v1/cursos/{id}`	`hasRole('ADMINISTRADOR')`	`204 No Content` / `404`

Con `02-plan.md` aprobado, el agente ya no tiene que "adivinar" nombres de campos, rutas ni códigos de estado: solo traduce el plano en código.

04. Desglose Atómico (03-tasks.md) y Ejecución Controlada



¿Por qué Dividir el Plan en Tareas Atómicas (03-tasks.md)?

01 • CERO EFECTO DOMINÓ

Aislamiento de Fallos

Si pides 5 clases en un solo prompt y un atributo del DTO queda mal tipado, el Mapper y el Controlador arrastrarán errores de compilación en cascada.

02 • ORDEN BOTTOM-UP

Grafo de Compilación

Construimos desde las hojas hacia la raíz: **T1 (DTOs) → T2 (Mapper) → T3 (Service) → T4 (Controller)**. Cada capa compila por sí sola.

03 • AUDITORÍA HUMANA

Revisión en 60 Segundos

Es 10 veces más rápido y seguro auditar 1 archivo de 40 líneas que inspeccionar 450 líneas generadas de golpe por el LLM.

04 • BITÁCORA EN GIT

Estado Persistente

El checklist `- [x]` dentro de `03-tasks.md` registra exactamente qué tareas ya fueron compiladas y cuáles faltan por ejecutar.

Universidad ICESI

El Mapa de 5 Tareas en 001-cursos-rest-api/03-tasks.md

Secuencia de Construcción Incremental (En Vivo en el IDE):

T1 — **Crear `CursoRequestDTO` y `CursoResponseDTO`**
Paquete `dto` con Lombok y Jakarta Validation. Verificación: `./mvnw test-compile`.

T2 — **Crear `CursoMapper` (Aplanamiento Seguro)**
Convierte entre `Curso`, `Profesor` y DTOs sin filtrar `Usuario.password`.

T3 — **Extender `CursoService.buscarPorDepartamento(...)`**
Delega en `cursoRepository.findByDepartamentoIgnoreCase` sin tocar MVC.

T4 — **Implementar `CursoRestController` + `@PreAuthorize`**
5 endpoints bajo `/api/v1/cursos` + Validación **T5** en Postman.

Invocación en el IDE con `/speckit.implement` (Single-Task Rule):

```
# Ejecutamos UNA sola tarea a la vez en el IDE:
/speckit.implement Ejecuta únicamente la Tarea T1 de
specs/001-cursos-rest-api/03-tasks.md.

## Protocolo que sigue el Agente en cada Tarea:
1. Lee los contratos:
   - specs/00-constitution.md
   - specs/001-cursos-rest-api/01-spec.md
   - specs/001-cursos-rest-api/02-plan.md
   - specs/001-cursos-rest-api/03-tasks.md

2. Implementa y Verifica:
   - Crea solo los archivos de T1 (CursoRequestDTO, CursoResponseDTO).
   - Ejecuta ./mvnw test-compile y marca - [x] Tarea T1.
```

Ahora pasamos al IDE: Ejecutaremos en vivo `T1 → T2 → T3 → T4` revisando cada archivo generado antes de probar en Postman.

05. Verificación en Postman y Automatización con /speckit.* y .agents

CIERRE DEL CICLO SDD: VALIDACIÓN EMPÍRICA Y AUTOMATIZACIÓN NATIVA

1. Escenarios Gherkin (01-spec.md)

Dado / Cuando / Entonces

`200 | 201 | 204 | 403 | 404`

Contrato de pruebas acordado

2. Validación Cruzada (Tarea T5)

Pruebas HTTP con Token JWT

`Postman / cURL + ./mvnw test`

Cierre de [x] en 03-tasks.md

3. Escalado a Nuevos Módulos

`.github/prompts/ (/speckit.*)`

`.agents/skills/ (speckit-*)`

Listo para 002-matriculas-rest-api

Universidad ICESI

Tarea T5: Trazabilidad Directa de 01-spec.md a Postman

Prueba	Escenario en 01-spec.md	Usuario Autenticado (JWT)	Petición HTTP	Respuesta Verificada
P1	**Escenario 1** (Catálogo filtrado)	`apaez@icesi.edu.co` (`ESTUDIANTE`)	`GET /api/v1/cursos?departamento=Computación y Sistemas Inteligentes`	`200 OK` Lista de 6 cursos TIC con `nombreProfesor` y `totalMatriculados`.
P2	**Escenario 2** (Rechazo RBAC)	`apaez@icesi.edu.co` (Sin `COURSE_WRITE`)	`POST /api/v1/cursos`	`403 Forbidden` Bloqueado por `@PreAuthorize` en formato JSON.
P3	**Escenario 3** (Registro exitoso)	`drincon@icesi.edu.co` (`PROFESOR`)	`POST /api/v1/cursos` (`profesorId: 3`)	`201 Created` + Header `Location: .../api/v1/cursos/15`.
P4	**Escenario 4** (Docente inexistente)	`drincon@icesi.edu.co` (`PROFESOR`)	`POST /api/v1/cursos` (`profesorId: 999`)	`404 Not Found` Captura limpia de `IllegalArgumentException`.
P5	**Regla RN-03** (Borrado Admin)	1° `drincon` (Profesor) → 2° `admin` (Admin)	`DELETE /api/v1/cursos/15`	1° `403 Forbidden` → 2° `204 No Content`.

Convergencia SDD: Recién cuando las 5 pruebas pasan en Postman y `./mvnw test` está en verde, marcamos `- [x] Tarea T5` en `03-tasks.md`.

Universidad ICESI

Checklist del Ingeniero al Auditar Código Generado por IA

01 • FRONTERA DTO

Privacidad y Ciclos

¿Algún método del controlador recibe o devuelve una clase `@Entity`? ¿Se filtran campos internos como `password` o colecciones perezosas?

02 • INYECCIÓN Y MAPEO

Estándar Lombok/MapStruct

¿Todas las dependencias son `private final` con `@RequiredArgsConstructor`? ¿El Mapper generó su clase `*MapperImpl` en compilación?

03 • SEGURIDAD RBAC

Alineación con data.sql

¿Cada endpoint tiene su anotación `@PreAuthorize` usando exactamente los nombres de permisos y roles definidos en `data.sql`?

04 • NO REGRESIÓN MVC

Convivencia Dual

¿Las vistas Thymeleaf existentes y los tests en `src/test/java` siguen funcionando sin haberse roto ningún método del `@Service`?

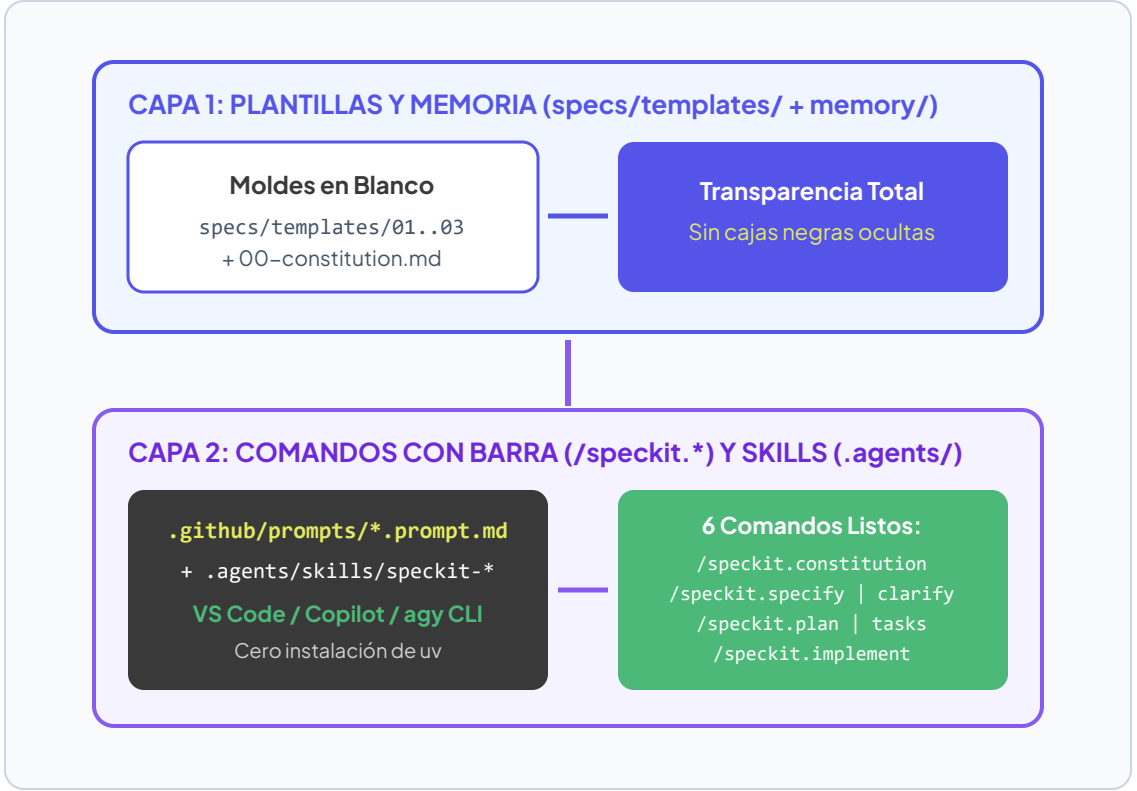
Cómo Funciona la Automatización: Slash Commands y .agents/

3 Piezas que Reemplazan a `uv` en Nuestro Repositorio:

1. `AGENTS.md` + `.agents/memory/` (Memoria Activa)
Conecta automáticamente `00-constitution.md`, `tech-stack.md` y `architecture.md` para que el agente nunca olvide la arquitectura del curso.

2. `.github/prompts/` (Slash Commands `/speckit.*`)
Habilitados en `.vscode/settings.json` (`chat.promptFiles: true`). Al escribir `/speckit.specify` o `/speckit.clarify`, el IDE inyecta el flujo SDD completo.

3. `.agents/skills/` (Skills Nativos en `agy` CLI)
Empaqueta las 6 fases como `SKILL.md` autodescubiertos por Antigravity CLI e IDE sin instalar librerías externas.



Catálogo de Comandos SDD: .github/prompts/ y .agents/skills/

FASE 1 Y 2 • SPECIFY

`/speckit.specify`

También: `/speckit.constitution` y skill `speckit-specify`

Toma tu requerimiento en lenguaje natural + `01-spec-template.md` y crea `specs/<id>/01-spec.md` con escenarios Gherkin.

FASE 3 • CLARIFY

`/speckit.clarify`

Skill: `.agents/skills/speckit-clarify/SKILL.md`

Ejecuta el **Interrogatorio Socrático**: formula 3 a 5 preguntas sobre casos borde y actualiza `01-spec.md` con tus respuestas.

FASE 4 Y 5 • PLAN & TASKS

`/speckit.plan & .tasks`

Skills: `speckit-plan` y `speckit-tasks`

Audita el código existente para diseñar `02-plan.md` (DTOs, Mappers, Endpoints) y lo divide en tareas atómicas en `03-tasks.md`.

FASE 6 • IMPLEMENT

`/speckit.implement`

Skill: `.agents/skills/speckit-implement/SKILL.md`

Aplica la **Single-Task Rule**: implementa una única tarea pendiente a la vez, verifica con `./mvnw test-compile` y marca `[x]`.

Reto en Vivo: Creando 002-matriculas-rest-api con /speckit.*

Flujo Completo de 6 Fases para tu Siguiente Módulo:

```
# 1. Especificar desde el template (Fase 2)
/speckit.specify Quiero crear 002-matriculas-rest-api en
/api/v1/matriculas para consultar matrículas e inscribir
estudiantes en cursos usando ENROLLMENT_WRITE.

# 2. Interrogatorio Socrático de casos borde (Fase 3)
/speckit.clarify Hazme las preguntas de casos borde sobre
matrículas duplicadas, cupos o semestres.

# 3. Diseño Técnico y Checklist Atómico (Fases 4 y 5)
/speckit.plan Genera 02-plan.md para 002-matriculas-rest-api
/speckit.tasks Genera 03-tasks.md para 002-matriculas-rest-api

# 4. Implementación Controlada Tarea por Tarea (Fase 6)
/speckit.implement Ejecuta únicamente la Tarea T1
```

Los 3 Mandamientos del Desarrollador SDD:

1. Gobernar y Clarificar antes de Diseñar
Nunca pases al plan técnico sin haber ejecutado `/speckit.clarify` para resolver casos borde en `01-spec.md`.

2. Auditar el Plan Técnico (`02-plan.md`)
El desarrollador es el arquitecto: revisa que la IA reutilice los servicios existentes y aplane los DTOs sin filtrar entidades.

3. Single-Task Rule y Verificación Empírica
Avanza una tarea a la vez en `03-tasks.md` verificando siempre con `./mvnw` y Postman.